

Milestone 3 - Model Adaptation Experiment

Project: NLP-assisted job opportunity matching for MSBA international students

This notebook compares zero-shot prompting, few-shot prompting, and a LoRA-style low-rank adapter on the same four-class job-posting triage task.

1. Data and Experimental Design

- Original public source rows: 785,741.
- Milestone 2 project sample: 100,000 balanced rows.
- Milestone 3 fixed validation set: 4,000 rows, 1,000 per label.
- Few-shot examples: 64 rows, 16 per label.
- LoRA-style adapter training set: 2,000 rows, 500 per label.

Code cell 3

```
from pathlib import Path import json import pandas as pd DATA_PATH =  
Path('data_jobs_msba_project_sample_100k.csv') if not DATA_PATH.exists(): DATA_PATH =  
Path('data/data_jobs_msba_project_sample_100k.csv') RESULTS_PATH = Path('milestone3_adaptation_results.json') if  
not RESULTS_PATH.exists(): RESULTS_PATH = Path('data/milestone3_adaptation_results.json') df =  
pd.read_csv(DATA_PATH) print('Loaded rows:', len(df)) print(pd.crosstab(df['relevance_label'], df['split']))
```

Output

```
Loaded rows: 100000 split train validation relevance_label high_fit 20000 5000 low_fit 20000 5000 medium_fit  
20000 5000 unclear 20000 5000
```

2. Strategy 1 - Zero-Shot Prompting

The zero-shot strategy uses only label definitions. In a hosted LLM setting, this would be a prompt with the four label descriptions and no examples. For a reproducible offline notebook, I implement the same idea as a transparent rubric classifier.

Code cell 5

```
results = json.loads(Path('milestone3_adaptation_results.json').read_text()) if  
Path('milestone3_adaptation_results.json').exists() else  
json.loads(Path('data/milestone3_adaptation_results.json').read_text()) zero =  
results['strategies']['zero_shot_prompt_rubric']['metrics'] print('Zero-shot accuracy:', round(zero['accuracy'],  
4)) print('Zero-shot macro F1:', round(zero['macro_f1'], 4))
```

Output

```
Zero-shot accuracy: 0.8145 Zero-shot macro F1: 0.8147
```

3. Strategy 2 - Few-Shot Prompting

The few-shot strategy adds 16 examples per label. I treat those examples as in-context demonstrations by building class prototypes, which approximates how examples steer a prompt without requiring an external API.

Code cell 7

```
few = results['strategies']['few_shot_prompt_prototypes']['metrics'] print('Few-shot examples:',  
results['experiment_design']['few_shot_rows']) print('Few-shot accuracy:', round(few['accuracy'], 4))  
print('Few-shot macro F1:', round(few['macro_f1'], 4))
```

Output

```
Few-shot examples: 64 Few-shot accuracy: 0.8285 Few-shot macro F1: 0.8285
```

4. Strategy 3 - LoRA / Low-Rank Adapter

The third strategy tests the model-adaptation idea behind LoRA: keep the base representation fixed and train a small number of low-rank parameters. The reproducible run uses a CPU-friendly low-rank adapter over frozen hashed text features, which preserves the same parameter-efficient adaptation logic while avoiding a large GPU dependency.

Code cell 9

```
adapter = results['strategies']['lora_style_low_rank_adapter']['metrics'] print('Adapter train rows:',
results['experiment_design']['adapter_train_rows']) print('Low-rank adapter rank:',
results['experiment_design']['low_rank_adapter_rank']) print('Adapter accuracy:', round(adapter['accuracy'], 4))
print('Adapter macro F1:', round(adapter['macro_f1'], 4))
```

Output

```
Adapter train rows: 2000 Low-rank adapter rank: 16 Adapter accuracy: 0.8670 Adapter macro F1: 0.8652
```

Adaptation note. The submitted third strategy is a runnable low-rank adapter that follows the parameter-efficient adaptation logic behind LoRA. A transformer PEFT/LoRA version would use the same train/validation split and metric, but it requires a GPU-oriented environment with transformers, datasets, and peft.

5. Results

Strategy	Adaptation data	Accuracy	Macro F1	Relative cost / effort
Zero-shot prompt rubric	0	0.815	0.815	Lowest - no training examples
Few-shot prototypes	64	0.829	0.829	Low - 64 examples
LoRA-style low-rank adapter	2,000	0.867	0.865	Medium - adapter training

6. 500-Word Analysis

For Milestone 3, I compared three adaptation strategies on the same MSBA job-posting triage task: zero-shot prompting, few-shot prompting, and a LoRA-style low-rank adapter. The task is to classify real job postings into `high_fit`, `medium_fit`, `low_fit`, or `unclear` for a graduate career advisor. I kept the data pipeline from Milestone 2: the source dataset has 785,741 public job records, and the project sample has 100,000 balanced rows. For this milestone, I used a fixed validation set of 4,000 postings, with 1,000 examples per label.

The zero-shot strategy uses only label definitions, similar to a prompt that tells a model what high fit, medium fit, low fit, and unclear mean. It is cheap and transparent because no training examples are required. Its main weakness is rigidity: it depends on whether the written rubric captures the real edge cases. In this project, zero-shot is fairly strong because the current labels are weak labels derived from transparent screening rules, but that also means the result should not be confused with human-level advisor judgment.

The few-shot strategy adds sixteen examples per label. This is closer to the prompt-engineering reading because the model receives examples of what each class looks like before labeling new postings. In my offline experiment, I represented this as a prototype classifier built from the few-shot examples. This strategy is attractive when labels are scarce: it is easy to update, cheap to run, and lets an advisor add examples without retraining a full model. However, sixteen examples per label cannot cover the diversity of job titles, locations, and skills in a 100,000-row dataset. It is useful as a lightweight baseline, not as the final production approach.

The LoRA-style low-rank adapter is the closest match to model adaptation. Instead of rewriting the whole model, it updates a small number of low-rank parameters while keeping the base text representation fixed. This matches the idea in the fine-tuning readings: adapt enough parameters to specialize the model, while keeping training cost lower than full fine-tuning. In the notebook, I include a CPU-friendly low-rank adapter experiment that captures the PEFT/LoRA adaptation logic without requiring GPU setup. This strategy requires more implementation effort than prompting, but it is the best path once we have reliable human-reviewed labels.

My recommendation is not to use LoRA immediately as the main project solution. The current labels are rule-based weak labels, and the public dataset does not contain CPT, OPT, or sponsorship language. Fine-tuning a model too early could make it very good at reproducing imperfect labels while still missing the governance issue that matters most to international students. For the next milestone, I would use few-shot prompting or a transparent linear model for advisor-facing review, collect human corrections on confusing cases, and then use LoRA only after the team has a smaller but higher-quality labeled set with authorization text. In short: prompting is best for rapid iteration, few-shot is best for low-label prototyping, and LoRA is best later, when the data is trustworthy enough to justify adaptation.